CSCI_6461
Computer System Architecture

Team 8
14th April, 2025

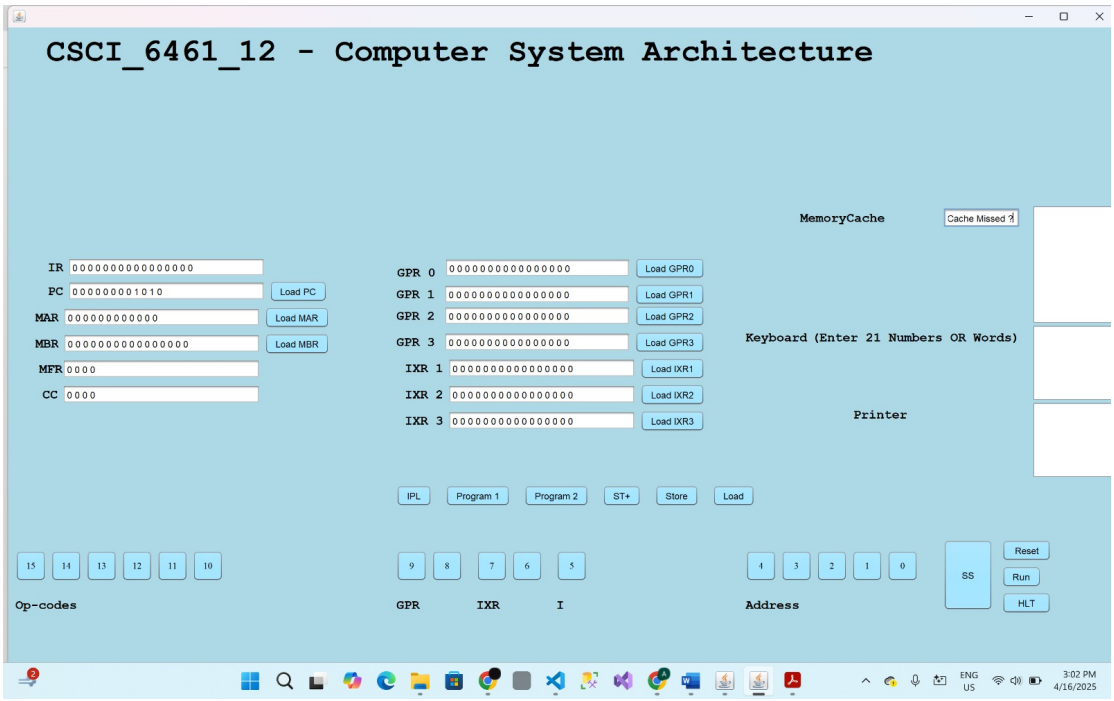
Project Part-3

Shrey Patel : G36586254
Anvay Paralikar : G41097732
Shanthan Reddy :G41758905

Design Notes

Items that Make up the Front Panel:-

The user can load values into the Instruction Register (IR), Memory Buffer Register (MBR), General Purpose Registers (GPR 0-3), Index Registers (IXR 1-3), Program Counter (PC), and Memory Address Register (MAR), all of which are 12 bits in size. Nevertheless, there is no load button on the four-bit Condition Code (CC) or the three-bit Memory Fault Register (MFR).



- **Program Counter (PC):-** The address of the command to be carried out is stored in the PC.
- **Instruction Register (IR):-** The instruction that the CPU is now executing is stored in IR.
- **Memory Address Register (MAR):-** This register keeps track of the address in memory that will be used to send or retrieve data.
- **Memory Buffer Register (MBR):-** The data being moved to or from memory is momentarily stored in the Memory Buffer Register (MBR).
- **Memory Fault Register (MFR):** Codes that denote particular memory-related problems or exceptions are stored in the
- **Condition Code (CC):** Flags that represent the results of operations, such as carry, overflow, zero, or negative, are stored in the Condition Code register.
- **General Purpose Registers (GPR 0-3):** During program execution, intermediate data and values are stored in these registers.
- **Index Registers (IXR 1-3):** During the execution of an instruction, the addresses of operands can be changed using the Index Registers.

Control Buttons:

- **IPL (Initial Program Load):** This button loads instructions from an external file (usually loading.txt) into memory to initialize the machine. The simulator reads the encoded instructions when this button is hit, adding the necessary information to the memory addresses. By making sure the simulator has the instructions it needs to process, this gets it ready for execution.
- **Programs 1 and 2:** This button enables you to upload a file and see how many words it contains.
- **ST+ (Step):** The user can carry out the program one instruction at a time by pressing the ST+ button. The simulator retrieves, decodes, and executes the current instruction from memory when this button is hit. Because it allows users to see how each instruction alters the state of the registers and memory, this step-by-step execution is helpful for troubleshooting and teaching.
- **Store:** To save a value from a designated register into a memory location, use the Store button. Pressing this button moves the contents of the register to the specified address in memory when the user specifies a register and a memory address. For saving interim results or data that must remain after the current execution cycle, this feature is crucial.
- **Load:** When manually updating the contents of different registers or memory locations, the front panel's load button is essential. It simulates the loading of data as part of the machine's function by enabling the user to enter certain values straight into a register or memory address. Clicking the Load Button will update that specific register or memory address with the new value once you have entered the appropriate value into the associated text field for that register. This will replicate a "load" operation as it would happen in the machine.
- **SS (Start/halt):** This button is a toggle that allows you to start or halt the loaded program's execution. When pressed, it starts carrying out the instructions one after the other until a halt condition is met or the button is hit one again to halt the process. With the use of this button, users may easily manage the simulation's flow and pause its execution for analysis or modifications.
- **Reset:** Pressing this button returns the simulator to its initial configuration, wiping off all registers and returning the RAM and program counter to their starting points. By doing this, users can begin anew without any leftover information or instruction states from earlier runs. It's very helpful for testing various scenarios or programs without having to restart the entire software.
- **Run:** Until a halt condition (HLT) is reached or the user chooses to halt the execution, the Run button runs the loaded program continuously from beginning to end. By simulating a program's whole execution cycle, users can gain insight into how a full set of instructions functions within the system.

- **HLT (Halt):** Pressing this button instantly halts the program's execution. When tapped, it instructs the simulator to stop all processes and stop executing any more commands. This button is essential for terminating a simulation session, especially in cases where unexpected behavior or an endless loop arises.

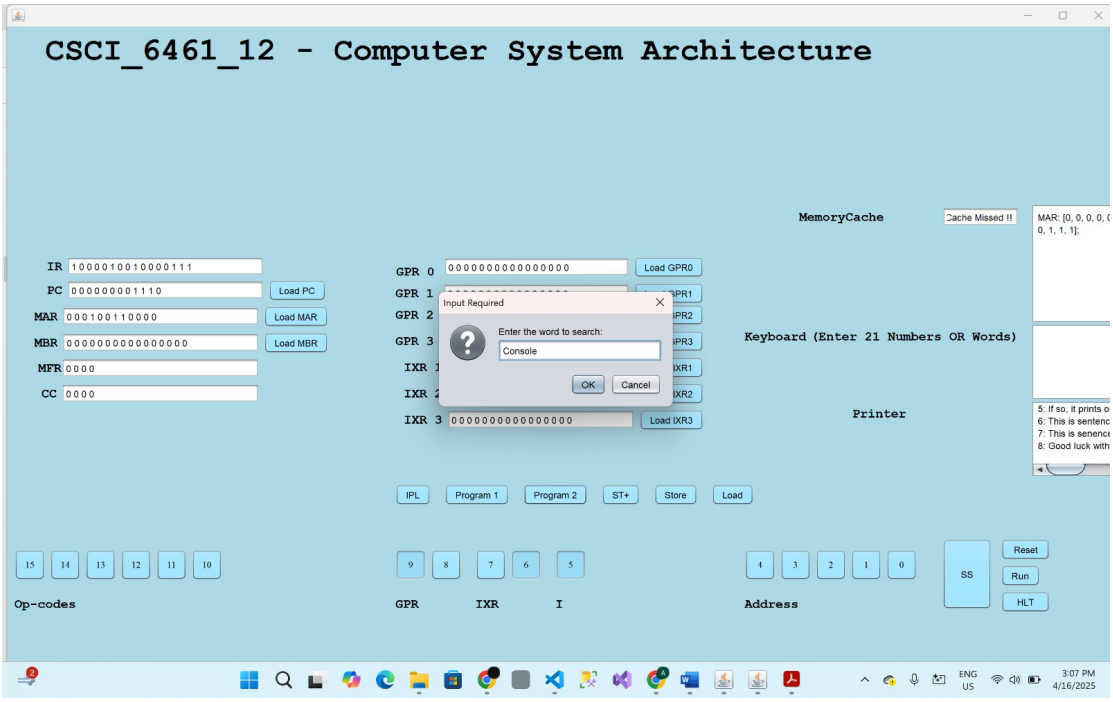
Cache Console:

The most recent MAR (Memory Address Register) command that was executed is shown in this cache console. Repeating the same instruction causes a cache hit; if not, a cache miss happens, and the console refreshes to reflect the most recent instruction.

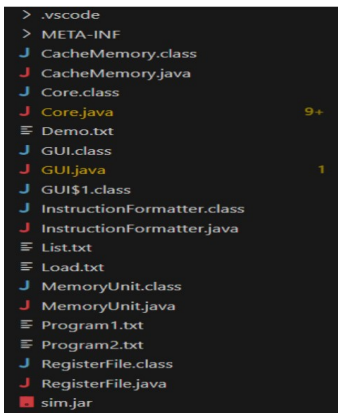
The cache employs a FIFO (First-In-First-Out) approach to make room for new instructions once it reaches its maximum capacity of 16 key-value pairs.

Keyboard and Printer:

Program Enter input is entered through the keyboard area, and the output is displayed in the printer terminal, using this keyboard and printer configuration.



Code Structure:



Core.java

The structure and operation of a simple processor emulator are specified by this Core.java file. It incorporates logic for carrying out instructions and builds important processor components like registers and memory.

Key Components:

- **Registers:** A number of registers with distinct sizes (bit lengths), including PC, CC, IR, MAR, MBR, MFR, GPR, and IX, are initialized. These contain crucial information for execution.
- **Memory:** To store values while an instruction is being executed, a memory object called MemoryUnit is created.

Important Functions:

1. execute(String type): This function mimics how an instruction would be carried out by the processor. After retrieving the instruction from memory and reading the program counter (PC), it decodes and runs the instruction. Opcodes such as LDR, STR, LDA, LDX, STX, and HLT are handled by it.

2. decodeOPCode(int[] binary_OPCode): The operation (LDR, STR, etc.) is determined by decoding the binary opcode from the instruction.

3. SetRegisterValue(String register, int[] value) and getRegisterValue(String register): Processor register values are retrieved and set by these functions.

4. Memory access is handled by getMemoryVals(int row) and setMemoryVals(int row, int[] value), which get and store values at designated memory addresses.

5. computeEA(int[] instruction): Determines an instruction's effective address (EA) while accounting for indexed and indirect addressing modes.

6. loadIPLFile(String path): Initializes the contents of memory based on input and loads a program into memory from a file.

7. The function openFileChooser() offers a user interface for choosing which file to put into memory.

The class mimics fundamental memory management and fetch-decode fetch-execute cycles, among others processor actions.

RegisterFile.java:

The Register class provides methods for interaction while simulating a Processor register by encapsulating its size and value. The main elements and their functions are as follows:

Key Components:

Variables by instance:

a) private int[] registerValue: An array containing the register's current value.
b) private int registerSize: An integer that indicates the register's size (in bits).

Constructor:

a) public Register(int size): Calls the initRegisterSize(size) function to initialize the register with a given size.

Getter Techniques:

a) The current value that is stored in the register is returned by the public int[] getRegVals() function.

private void init(int size): The size of the register is returned by the private void init(int size) function.

The Setter Method;

a) public void setRegisterVals(int[] newValue): If the size is not zero, This function sets the register's value. An Illegal State Exception is thrown if the register size is 0. Arrays are used to copy the new value into the register.copyOf to guarantee that changes made to the input array don't impact the internal state of the register.

MemoryUnit.java

The MemoryUnit class mimics a processor's memory structure, which consists of 2048 words with 16 bits each. It offers methods for setting (setMemoryVals) and retrieving (getMemoryVals) memory values while using array copying to maintain data integrity. The class is an essential part of processor simulation since it encapsulates the memory representation and provides secure access to its contents.

InstructionFormatter.java:-

Utility functions for converting hexadecimal strings to integers and binary arrays to integers are provided by the InstructionFormatter class. It simplifies the process of converting hexadecimal values to 16-bit and 12-bit binary arrays using public wrapper methods by introducing a private helper method called hexToBinArr that takes a given length for the binary array. This preserves the original functionality while improving code reusability and maintainability.

Load.txt

An Initial Program Load (init) button on the machine's front interface loads the loading.txt file into memory and restarts the computer. Each line of the hexadecimal file has two hexadecimal numbers: the first represents the machine address, and the second represents the material located at that location.

GUI.java

- This file contains the entire simulator's user interface, including all of the buttons, switches, and register displays. It adds on-click action for each component, changes memory, and carries out the instructions. It is essentially the primary driver of the project.
- **Functions:-**
  - loadST()- Loads the memory [MAR] = [MBR]
  - StorePlusButtonClick - StorePlusButtonClick loads the
  - memory[MAR] =MBR and automatically increases the MAR value.
  - HaltButtonClick - HaltButtonClick stops the program from starting.
  - initButtonClcik - loads the pre-program into the memory.
  - UltimateLoadAction - UltimateLoadAction runs instructions stored at memory[PC].
  - ResetButtonClick - Clicking the ResetButton completely clears the memory.

CacheMemory.java:-

The ` CacheMemory ` class employs the Least Recently Used (LRU) technique to create a simple, fixed-size cache. Here is a thorough breakdown of its elements and goals.
A finite number of key-value pairs are intended to be stored in the cache. In accordance with the LRU policy, the oldest entry is deleted to make room for new data when it reaches its maximum capacity. This improves the efficiency of data retrieval by guaranteeing that frequently retrieved material stays available in memory.

Fields:-

i. cache: Key-value pairs are stored in a `ConcurrentHashMap`. Because it is thread-safe, several threads can communicate with it without requiring additional synchronization.
ii. `capacity`: The most items that the cache is capable of storing.
iii. A `Deque` (double-ended queue) called `keysOrder` keeps track of the keys' order as they are added.This makes it easier to keep track of the least used things.

Constructor:-

i. sets the `cache`, `capacity`, and `keysOrder` to their initial values. The cache's maximum capacity is determined by the `capacity` option.

Methods:

i. The private helper method -`modifyKeyOrder(String key)` appends the key to the end of `keysOrder`. The oldest key is deleted from both `keysOrder` and `cache` if the cache grows larger than it can.
ii. `OrderOfKeys()`: Provides a string with commas between each key in `keysOrder`. This is helpful for logging or showing the contents of the current cache.
iii. `insertVal(String key, String value)`: Updates the value if the key already exists or adds a key-value pair to the cache. To guarantee that the cache size remains within its bounds, it invokes `modifyKeyOrder`.
iv. `removeVal(String key)`: Takes a key and its value out of `cache` and `keysOrder` simultaneously.
v. The function `fetchVal(String key)` returns the value that corresponds to the specified key. It returns `null` if the key is not in the cache.

Demo.txt:-

An upload value button on the machine's front interface launches the application. The instructions to launch the demo.txt file are in hexadecimal format.